AI 计算系统
实验操作手册

实验序号：八

实验名称：全流程垃圾分类应用

教　　师：朱光辉

学　　校：南京大学

时　　间：2022 年 8 月 23 日

八 全流程垃圾分类应用

8.1 实验目的

本实验的目的是熟悉图片分类从训练到部署的全流程应用，熟悉和使用ModelArts平台和MindSpore深度学习框架，掌握移植和优化代表性图片分类算法（SENet）的方法和流程，基于垃圾分类数据集实现图片分类算法的训练、验证、推理和部署。

- 基于Modelarts平台，使用MindSpore框架构建图片分类网络，在垃圾分类训练数据集上完成模型训练，并保存对应的模型权重。
- 基于Modelarts平台，使用MindSpore框架导入训练好的图片分类网络，在垃圾分类测试数据集上完成模型验证和推理。
- 使用AscendCL实现模型应用的部署开发，搭建一个实时的图片分类应用。

8.2 背景介绍

8.2.1 图片分类

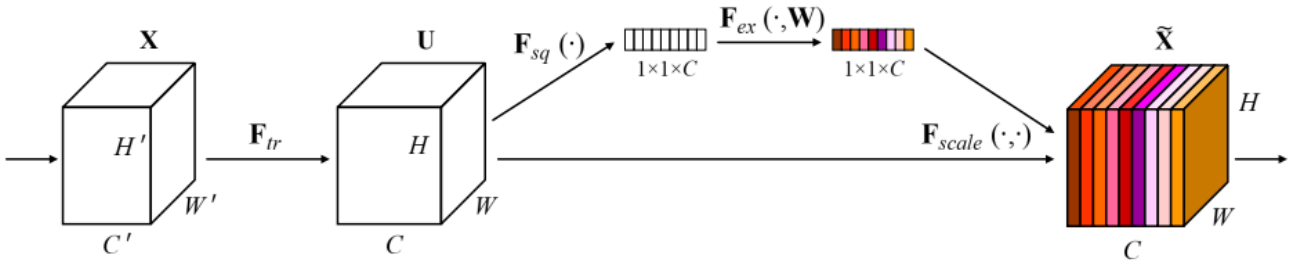
图像分类是根据图像的语义信息对不同类别图像进行区分，是计算机视觉中重要的基础问题，是目标检测、图像分割、人脸识别等其它高层次视觉任务的基础。图像分类在许多领域都有着广泛的应用，如：农业领域的植物健康状态分类，交通领域的车辆型号分类，互联网领域基于内容的相册自动归类，医学领域的病灶分类等。

基于深度学习的图片分类算法通常有两阶段构成：第一阶段称为特征提取阶段，主要是通过卷积神经网络提取图片的高纬度特征；第二阶段称为预测阶段，通常通过全连接层进一步提取特征，最后输出对应类别的概率。

8.2.2 SENet

SENet（Squeeze-and-Excitation Networks ）是 ImageNet 2017 竞赛 Image Classification 任务的冠军。在ImageNet数据集上将top-5 错误率降低到2.251%，原先的最好成绩是2.991%。SENet网络创新点在于关注通道之间的关系，希望模型可以自动学习到不同通道特征的重要程度，也就是如下图所示的Squeeze-and-Excitation Module。

SE模块首先对卷积得到的特征图进行Squeeze操作，得到channel的全局特征，然后对全局特征进行Excitation操作，学习各个channel之间的关系，也得到不同的channel的权重，最后乘以原来的特征图得到加权特征。本质上，SE模块是channel维度上做attention或者gating操作，使模型可以更加关注更具有表征能力的通道特征，抑制那些不重要的通道特征。



1.Squeeze-and-Excitation Module

SE模块主要包括Squeeze和Excitation两个操作，可以适用任何映射$F_{tr}: X \to U, X \in R^{H' \times W' \times C'}, U \in R^{H \times W \times C}$，以卷积为例，卷积核为$V = [v_1, v_2, \ldots, v_c]$，其中$v_c$代表第c个卷积核，那么输出$U = [u_1, u_2, \ldots, u_c]: u_c = v_c * X = \sum_{s=1}^{C'} v_c^s * x^s$，其中*代表卷积操作，而$v_c^s$代表一个s通道的二维卷积核，其输入一个通道上的空间特征，它学习特征空间关系，但是由于对各个通道的卷积结果做了求和，所以通道特征关系与卷积核学习到的空间关系混合在一起。而SE模块就是为了抽离这种混杂，使得模型直接学习到通道特征关系。

2.Squeeze操作

由于卷积只是在一个局部空间内进行操作，U很难获得足够的信息来提取通道之间的关系，对于网络中前面的层这更严重，因为感受野比较小。为了，SENet提出Squeeze操作，将一个通道上整个空间特征编码为一个全局特征，采用global average pooling来实现：

$$z_c = F_{sq}(u_c) = \frac{1}{H \times W} \sum_{i=1}^{H} \sum_{j=1}^{W} u_c(i,j), z \in R^c$$

3.Excitation操作

Sequeeze操作得到了全局描述特征，接下来需要另外一种运算来抓取通道之间的关系。这个操作需要满足两个准则：首先要灵活，它要可以学习到各个channel之间的非线性关系；第二点是学习的关系不是互斥的，因为这里允许多channel特征，而不是one-hot形式。基于此，这里采用sigmoid形式的gating机制：

$$s = F_{ex}(z, W) = \sigma(g(z, W)) = \sigma(W_2 ReLU(W_1 z))$$

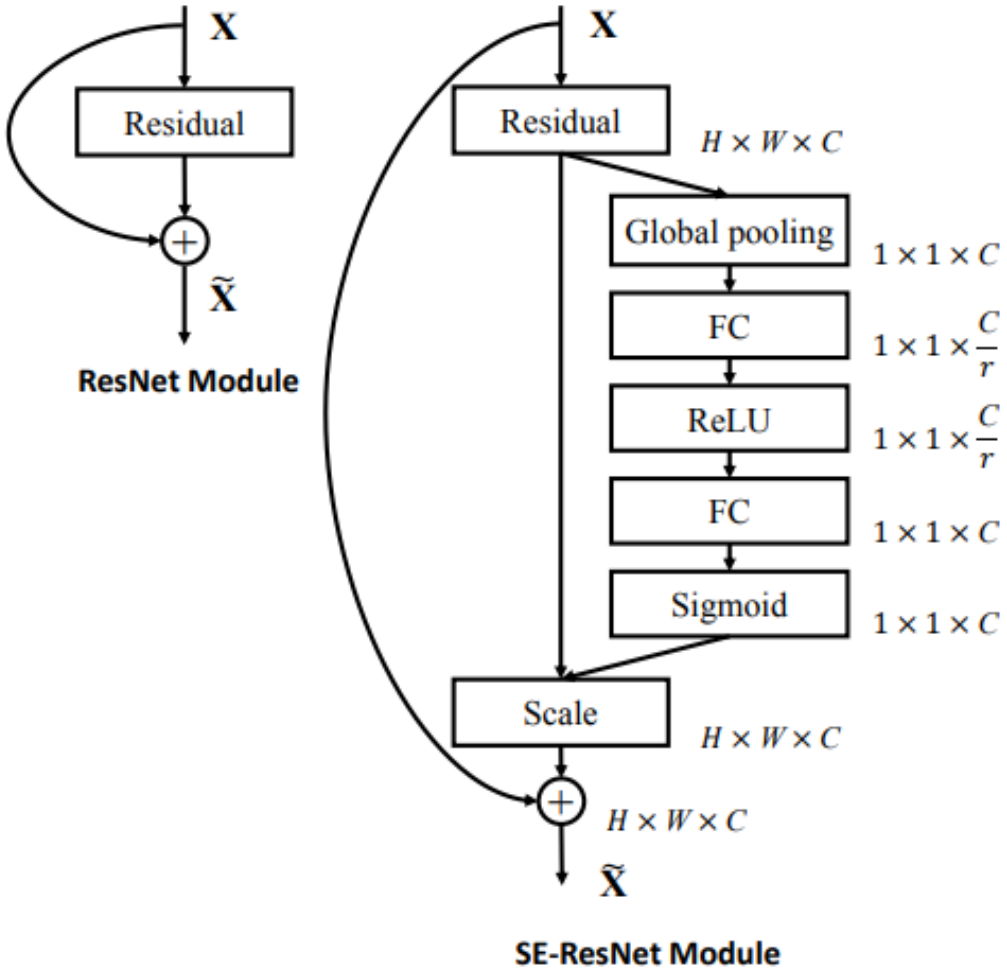
其中$W_1 \in R^{\frac{C}{r} \times C}, W_2 \in R^{\frac{C}{r} \times C}$。为了降低模型复杂度以及提升泛化能力，这里采用包含两个全连接层的bottleneck结构，其中第一个FC层起到降维的作用，降维系数为r是个超参数，然后采用ReLU激活。最后的FC层恢复原始的维度。最后将学习到的各个通过的激活值乘以U上的原始特征：

$$\hat{x}c = F_{scale}(u_c, s_c) = s_c \cdot u_c$$

其实整个操作可以看成学习到了各个通道的权重系数，从而使得模型对各个通道的特征更有辨别能力，这应该也算一种注意力机制。

4.SE模块在ResNet上的应用

SE模块的灵活性在于它可以直接应用现有的网络结构中。对于ResNet，SE模块嵌入到残差结构中的残差学习分支中。具体如下图所示：



5.SEResNet-50

此实验使用SEResNet-50网络实现垃圾分类应用，具体网络结构如下所示

Input Size	SE-ResNet-50	Output Size
3×224×224	[conv, 7 × 7, 64, stride 2], [max pool, 3 × 3, stride 2]	64x112×112
64x112×112	[[conv, 1×1, 64], [conv, 3×3, 64], [conv, 1×1, 256] [fc, 16, 256] × 3	256x56×56
256x56×56	[[conv, 1×1, 128], [conv, 3×3, 128], [conv, 1×1, 512] [fc, 32, 512] × 4	512x28×28
512x28×28	[[conv, 1×1, 256], [conv, 3×3, 256], [conv, 1×1, 1024] [fc, 64, 1024] × 6	1024x14×14
1024x14×14	[[conv, 1×1, 512], [conv, 3×3, 512], [conv, 1×1, 2048] [fc, 128, 2048] × 3	20487×7
2048x7×7	global average pool	2048
2048	40-d fc, softmax	40

8.3 实验环境

平台：Modelarts

操作系统：euler（aarch64）

软件环境：编程框架MindSpore1.7.1、异构计算架构CANN5.1，Python3.7.10

硬件环境：Ascend910A

数据集：垃圾分类数据集。该数据包括训练集、验证集和测试集，其中训练集包含8869张训练图片、验证集包含2978张验证图片，测试集包含2956张测试图片，共40个类别。该数据集的每个类别下的训练图片数目不同以及尺寸不一，需要再使用时做预处理操作。

8.4 实验内容

1.数据集分析：首先对垃圾分类数据集进行简单的数据分析，查看数据分布状况以及数据状态。

2.模型训练：使用MindSpore框架构建图片分类算法SEResNet-50，使用垃圾分类数据集在Ascend910加速卡上进行模型训练

3.模型验证：在测试集上测试模型的鲁棒性、准确率并获取性能指标

4.模型导出：将训练好的模型进行导出，使之脱离MindSpore框架

5.模型部署：将训练的模型部署并打包成应用，构建单张图片推理脚本，端到端的进行图片推理

8.5 实验步骤

8.5.1 数据分析

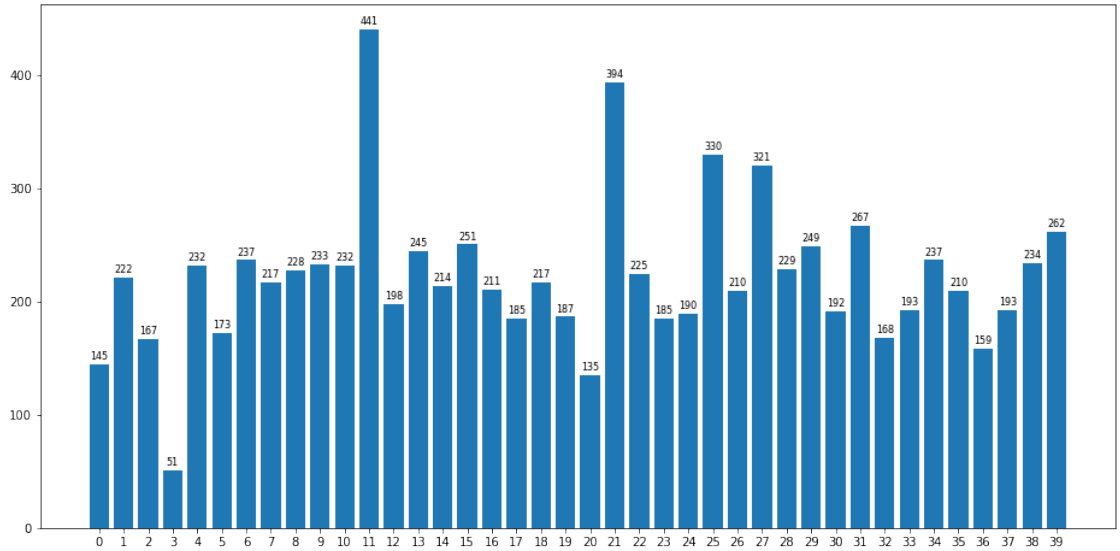
数据的好坏决定了模型的上下限，对于基本的图片分类任务，首先需要对数据集做一个数据分析，查看数据分布状况以及数据质量状态，并针对不同的数据集状态选择不同的处理方式，从而达到提升模型性能的效果。

这里统计训练集下每个类别下的图片数量关系以及训练集下图片大小的分布状态，代码如下：

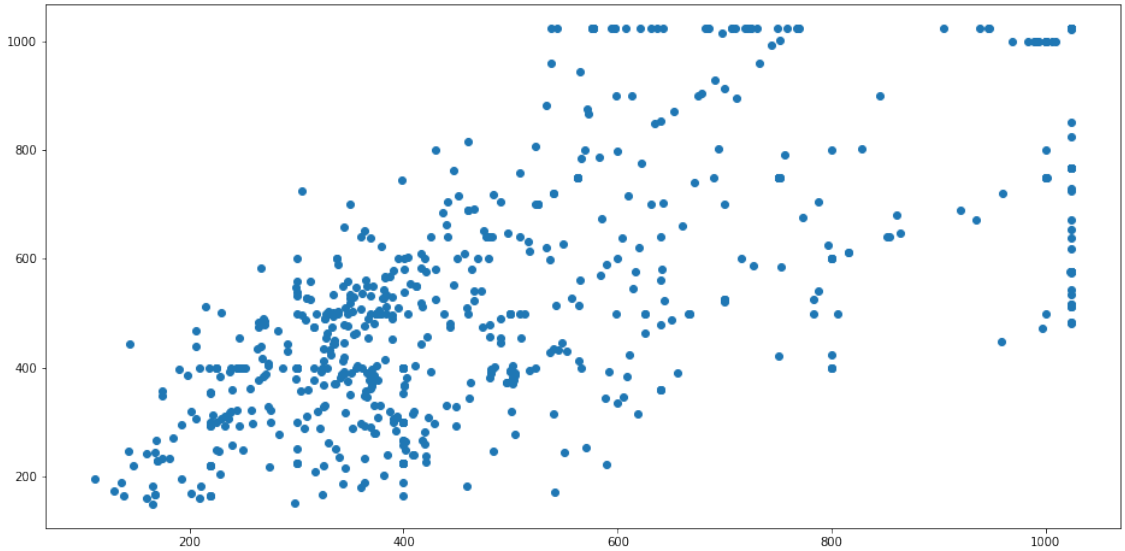
```
import os
import numpy as np
import random
from tqdm import tqdm
import cv2
from matplotlib import pyplot as plt

def draw_data_count(image_dir, dtype="bar"):
    sub_folders = next(os.walk(image_dir))[1]
    sub_folders.sort(key=lambda x:int(x))
    sub_folders_info = []
    all_image_nums = 0
    for sub_folder in tqdm(sub_folders):
        image_ids = next(os.walk(os.path.join(image_dir, sub_folder)))[2]
        image_ids = [i for i in image_ids]
        image_counts = len(image_ids)
        sub_folders_info.append((sub_folder, image_counts))
```

```
        all_image_nums += image_counts

    x = [i[0] for i in sub_folders_info]
    y = [i[1] for i in sub_folders_info]
    plt.figure(figsize=(16, 8))

    if dtype == "bar":
        plt.bar(x, y, width=0.8)
        for i,j in enumerate(y):
            plt.text(i, j+4, j,ha='center',fontsize=8)
    elif dtype == "pie":
        plt.pie(y, labels=x)
        plt.axis('equal')
    else:
        raise ValueError

def draw_data_size(image_dir):
    sub_folders = next(os.walk(image_dir))[1]
    sub_folders.sort(key=lambda x:int(x))
    data_size_list = []
    for sub_folder in tqdm(sub_folders):
        image_ids = next(os.walk(os.path.join(image_dir, sub_folder)))[2]
        # 这里随机取样十分之一样本统计尺寸大小
        random.shuffle(image_ids)
        image_shapes = [cv2.imread(os.path.join(image_dir, sub_folder,
image_ids[i])).shape[:2] for i in range(len(image_ids)//10)]
        data_size_list.extend(image_shapes)

    h = [i[0] for i in data_size_list]
    w = [i[1] for i in data_size_list]
    plt.figure(figsize=(16, 8))
    plt.scatter(h, w)

# 统计训练集每个类别的数量
draw_data_count("../data/train/", dtype="bar")
plt.show()
# 统计训练集尺寸大小分布
draw_data_size("../data/train/")
plt.show()
```

40个类别的训练图片数量如下所示，可以看出40个类别中各个数据量比较均匀，但类别为"3"的数据量比较少，在训练过程中可能由于数据量不足导致对类别3的识别效果较差，若结果符合预期，则可以采用增加类别3样本的手段，如数据扩充，数据增强等，提升模型对类别3样本的识别效果。



训练图片的尺寸大小如下所示，可以看出图片的尺寸大小不一，需要对图片做缩放操作后再送入神经网络进行训练。图片尺寸是决定网络模型的一个重要因素，图片尺寸大则具有更多的信息量，能够融合更多信息；图片尺寸小则具有更快的训练速度。



8.5.2 模型训练

实验四已经详细介绍了modelarts平台的使用和各个模块，其主要流程为：数据加载、模型搭建、设置优化器以及对应超参数。其主函数如下所示：

- 模型加载

```
def resnet_create_dataset(data_home, image_size, batch_size, rank_id=0, rank_size=1,
training=True,valid=False):
    """Data operations."""
    data_dir = data_home
    data_dir = os.path.join(data_home, "train")
    if not training:
        data_dir = os.path.join(data_home, "test")
    if valid:
        data_dir = os.path.join(data_home, "val")
    label_list={}
    for i in range(40):
        label_list[str(i)]=i
    data_set = de.ImageFolderDataset(data_dir,
                                     class_indexing=label_list,
                                     shuffle=False, num_shards=rank_size,
shard_id=rank_id)

    transform_img = vision.RandomCropDecodeResize([288,288], scale=(0.08, 1.0),
                                                  ratio=(0.75, 1.333))  # 改变尺寸
    rescale = 1.0 / 255.0
    shift = 0.0
    changeswap_op = vision.HWC2CHW()
    type_cast_op = C.TypeCast(mstype.float32)
    resize_op = vision.Resize(image_size)  # interpolation default BILINEAR
    rescale_op = vision.Rescale(rescale, shift)
    random_horizontal_op = vision.RandomHorizontalFlip()
    normalize_op =  vision.Normalize((0.4465, 0.4822, 0.4914), (0.2010, 0.1994,
0.2023))

    # apply map operations on images
     #data_set = data_set.map(operations=type_cast_op, input_columns="label")
    if training:
        data_set = data_set.map(input_columns="image", operations=transform_img)
        data_set = data_set.map(input_columns="image", operations=random_horizontal_op)
    else:
        decode_op = vision.Decode()
        data_set = data_set.map(input_columns="image", operations=decode_op)
        data_set = data_set.map(input_columns="image", operations=resize_op)

    data_set = data_set.map(input_columns="image", operations=normalize_op)
    data_set = data_set.map(input_columns="image", operations=type_cast_op)
    data_set = data_set.map(input_columns="image", operations=changeswap_op)

    # apply shuffle operations
    data_set = data_set.shuffle(buffer_size=data_set.get_dataset_size())

    # apply batch operations
    data_set = data_set.batch(batch_size=batch_size, drop_remainder=True)
```

```
    return data_set
```

- 模型搭建

```
import math
import numpy as np
from scipy.stats import truncnorm
import mindspore.nn as nn
import mindspore.ops as ops
import mindspore.common.dtype as mstype
from mindspore.common.tensor import Tensor


def conv_variance_scaling_initializer(in_channel, out_channel, kernel_size):
    fan_in = in_channel * kernel_size * kernel_size
    scale = 1.0
    scale /= max(1., fan_in)
    stddev = (scale ** 0.5) / .87962566103423978

    mu, sigma = 0, stddev
    weight = truncnorm(-2, 2, loc=mu, scale=sigma).rvs(out_channel * in_channel *
kernel_size * kernel_size)
    weight = np.reshape(weight, (out_channel, in_channel, kernel_size, kernel_size))
    return Tensor(weight, dtype=mstype.float32)


def _weight_variable(shape, factor=0.01):
    init_value = np.random.randn(*shape).astype(np.float32) * factor
    return Tensor(init_value)


def calculate_gain(nonlinearity, param=None):
    """calculate_gain"""
    linear_fns = ['linear', 'conv1d', 'conv2d', 'conv3d', 'conv_transpose1d',
'conv_transpose2d', 'conv_transpose3d']
    res = 0
    if nonlinearity in linear_fns or nonlinearity == 'sigmoid':
        res = 1
    elif nonlinearity == 'tanh':
        res = 5.0 / 3
    elif nonlinearity == 'relu':
        res = math.sqrt(2.0)
    elif nonlinearity == 'leaky_relu':
        if param is None:
            neg_slope = 0.01
        elif not isinstance(param, bool) and isinstance(param, int) or
isinstance(param, float):
```

```
            neg_slope = param
        else:
            raise ValueError("neg_slope {} not a valid number".format(param))
        res = math.sqrt(2.0 / (1 + neg_slope ** 2))
    else:
        raise ValueError("Unsupported nonlinearity {}".format(nonlinearity))
    return res


def _calculate_fan_in_and_fan_out(tensor):
    """_calculate_fan_in_and_fan_out"""
    dimensions = len(tensor)
    if dimensions < 2:
        raise ValueError("Fan in and fan out can not be computed for tensor with fewer
than 2 dimensions")
    if dimensions == 2:  # Linear
        fan_in = tensor[1]
        fan_out = tensor[0]
    else:
        num_input_fmaps = tensor[1]
        num_output_fmaps = tensor[0]
        receptive_field_size = 1
        if dimensions > 2:
            receptive_field_size = tensor[2] * tensor[3]
        fan_in = num_input_fmaps * receptive_field_size
        fan_out = num_output_fmaps * receptive_field_size
    return fan_in, fan_out


def _calculate_correct_fan(tensor, mode):
    mode = mode.lower()
    valid_modes = ['fan_in', 'fan_out']
    if mode not in valid_modes:
        raise ValueError("Unsupported mode {}, please use one of {}".format(mode,
valid_modes))
    fan_in, fan_out = _calculate_fan_in_and_fan_out(tensor)
    return fan_in if mode == 'fan_in' else fan_out


def kaiming_normal(inputs_shape, a=0, mode='fan_in', nonlinearity='leaky_relu'):
    fan = _calculate_correct_fan(inputs_shape, mode)
    gain = calculate_gain(nonlinearity, a)
    std = gain / math.sqrt(fan)
    return np.random.normal(0, std, size=inputs_shape).astype(np.float32)


def kaiming_uniform(inputs_shape, a=0., mode='fan_in', nonlinearity='leaky_relu'):
    fan = _calculate_correct_fan(inputs_shape, mode)
    gain = calculate_gain(nonlinearity, a)
```

```
    std = gain / math.sqrt(fan)
    bound = math.sqrt(3.0) * std  # Calculate uniform bounds from standard deviation
    return np.random.uniform(-bound, bound, size=inputs_shape).astype(np.float32)


def _conv3x3(in_channel, out_channel, stride=1, use_se=False, res_base=False):
    if use_se:
        weight = conv_variance_scaling_initializer(in_channel, out_channel,
kernel_size=3)
    else:
        weight_shape = (out_channel, in_channel, 3, 3)
        weight = Tensor(kaiming_normal(weight_shape, mode="fan_out",
nonlinearity='relu'))
    if res_base:
        return nn.Conv2d(in_channel, out_channel, kernel_size=3, stride=stride,
                         padding=1, pad_mode='pad', weight_init=weight)
    return nn.Conv2d(in_channel, out_channel, kernel_size=3, stride=stride,
                     padding=0, pad_mode='same', weight_init=weight)


def _conv1x1(in_channel, out_channel, stride=1, use_se=False, res_base=False):
    if use_se:
        weight = conv_variance_scaling_initializer(in_channel, out_channel,
kernel_size=1)
    else:
        weight_shape = (out_channel, in_channel, 1, 1)
        weight = Tensor(kaiming_normal(weight_shape, mode="fan_out",
nonlinearity='relu'))

    if res_base:
        return nn.Conv2d(in_channel, out_channel, kernel_size=1, stride=stride,
                         padding=0, pad_mode='pad', weight_init=weight)
    return nn.Conv2d(in_channel, out_channel, kernel_size=1, stride=stride,
                     padding=0, pad_mode='same', weight_init=weight)


def _conv7x7(in_channel, out_channel, stride=1, use_se=False, res_base=False):
    if use_se:
        weight = conv_variance_scaling_initializer(in_channel, out_channel,
kernel_size=7)
    else:
        weight_shape = (out_channel, in_channel, 7, 7)
        weight = Tensor(kaiming_normal(weight_shape, mode="fan_out",
nonlinearity='relu'))

    if res_base:
        return nn.Conv2d(in_channel, out_channel,
                         kernel_size=7, stride=stride, padding=3, pad_mode='pad',
weight_init=weight)
```

```
    return nn.Conv2d(in_channel, out_channel,
                     kernel_size=7, stride=stride, padding=0, pad_mode='same',
weight_init=weight)


def _bn(channel, res_base=False):
    if res_base:
        return nn.BatchNorm2d(channel, eps=1e-5, momentum=0.1,
                              gamma_init=1, beta_init=0, moving_mean_init=0,
moving_var_init=1)
    return nn.BatchNorm2d(channel, eps=1e-4, momentum=0.9,
                          gamma_init=1, beta_init=0, moving_mean_init=0,
moving_var_init=1)


def _bn_last(channel):
    return nn.BatchNorm2d(channel, eps=1e-4, momentum=0.9,
                          gamma_init=0, beta_init=0, moving_mean_init=0,
moving_var_init=1)


def _fc(in_channel, out_channel, use_se=False):
    if use_se:
        weight = np.random.normal(loc=0, scale=0.01, size=out_channel * in_channel)
        weight = Tensor(np.reshape(weight, (out_channel, in_channel)),
dtype=mstype.float32)
    else:
        weight_shape = (out_channel, in_channel)
        weight = Tensor(kaiming_uniform(weight_shape, a=math.sqrt(5)))

    return nn.Dense(in_channel, out_channel, has_bias=True, weight_init=weight,
bias_init=0)


class ResidualBlock(nn.Cell):

    expansion = 4

    def __init__(self,
                 in_channel,
                 out_channel,
                 stride=1,
                 use_se=False, se_block=False):
        super(ResidualBlock, self).__init__()
        self.stride = stride
        self.use_se = use_se
        self.se_block = se_block
        channel = out_channel // self.expansion
        self.conv1 = _conv1x1(in_channel, channel, stride=1, use_se=self.use_se)
```

```
        self.bn1 = _bn(channel)
        if self.use_se and self.stride != 1:
            self.e2 = nn.SequentialCell([_conv3x3(channel, channel, stride=1,
use_se=True), _bn(channel),
                                         nn.ReLU(), nn.MaxPool2d(kernel_size=2,
stride=2, pad_mode='same')])
        else:
            self.conv2 = _conv3x3(channel, channel, stride=stride, use_se=self.use_se)
            self.bn2 = _bn(channel)

        self.conv3 = _conv1x1(channel, out_channel, stride=1, use_se=self.use_se)
        self.bn3 = _bn(out_channel)

        if self.se_block:
            self.se_global_pool = ops.ReduceMean(keep_dims=False)
            self.se_dense_0 = _fc(out_channel, int(out_channel / 4),
use_se=self.use_se)
            self.se_dense_1 = _fc(int(out_channel / 4), out_channel,
use_se=self.use_se)
            self.se_sigmoid = nn.Sigmoid()
            self.se_mul = ops.Mul()
        self.relu = nn.ReLU()

        self.down_sample = False

        if stride != 1 or in_channel != out_channel:
            self.down_sample = True
        self.down_sample_layer = None

        if self.down_sample:
            if self.use_se:
                if stride == 1:
                    self.down_sample_layer = nn.SequentialCell([_conv1x1(in_channel,
out_channel,
                                                                         stride,
use_se=self.use_se), _bn(out_channel)])
                else:
                    self.down_sample_layer =
nn.SequentialCell([nn.MaxPool2d(kernel_size=2, stride=2, pad_mode='same'),
                                                                _conv1x1(in_channel,
out_channel, 1,

use_se=self.use_se), _bn(out_channel)])
            else:
                self.down_sample_layer = nn.SequentialCell([_conv1x1(in_channel,
out_channel, stride,

use_se=self.use_se), _bn(out_channel)])
```

```
    def construct(self, x):
        identity = x

        out = self.conv1(x)
        out = self.bn1(out)
        out = self.relu(out)
        if self.use_se and self.stride != 1:
            out = self.e2(out)
        else:
            out = self.conv2(out)
            out = self.bn2(out)
            out = self.relu(out)
        out = self.conv3(out)
        out = self.bn3(out)
        if self.se_block:
            out_se = out
            out = self.se_global_pool(out, (2, 3))
            out = self.se_dense_0(out)
            out = self.relu(out)
            out = self.se_dense_1(out)
            out = self.se_sigmoid(out)
            out = ops.reshape(out, ops.shape(out) + (1, 1))
            out = self.se_mul(out, out_se)

        if self.down_sample:
            identity = self.down_sample_layer(identity)

        out = out + identity
        out = self.relu(out)

        return out

class ResNet(nn.Cell):
    def __init__(self,
                 block,
                 layer_nums,
                 in_channels,
                 out_channels,
                 strides,
                 num_classes,
                 use_se=False,
                 res_base=False):
        super(ResNet, self).__init__()

        if not len(layer_nums) == len(in_channels) == len(out_channels) == 4:
            raise ValueError("the length of layer_num, in_channels, out_channels list
must be 4!")
        self.use_se = use_se
        self.res_base = res_base
```

```
        self.se_block = False
        if self.use_se:
            self.se_block = True

        if self.use_se:
            self.conv1_0 = _conv3x3(3, 32, stride=2, use_se=self.use_se)
            self.bn1_0 = _bn(32)
            self.conv1_1 = _conv3x3(32, 32, stride=1, use_se=self.use_se)
            self.bn1_1 = _bn(32)
            self.conv1_2 = _conv3x3(32, 64, stride=1, use_se=self.use_se)
        else:
            self.conv1 = _conv7x7(3, 64, stride=2, res_base=self.res_base)
        self.bn1 = _bn(64, self.res_base)
        self.relu = ops.ReLU()

        if self.res_base:
            self.pad = nn.Pad(paddings=((0, 0), (0, 0), (1, 1), (1, 1)))
            self.maxpool = nn.MaxPool2d(kernel_size=3, stride=2, pad_mode="valid")
        else:
            self.maxpool = nn.MaxPool2d(kernel_size=3, stride=2, pad_mode="same")

        self.layer1 = self._make_layer(block,
                                       layer_nums[0],
                                       in_channel=in_channels[0],
                                       out_channel=out_channels[0],
                                       stride=strides[0],
                                       use_se=self.use_se)
        self.layer2 = self._make_layer(block,
                                       layer_nums[1],
                                       in_channel=in_channels[1],
                                       out_channel=out_channels[1],
                                       stride=strides[1],
                                       use_se=self.use_se)
        self.layer3 = self._make_layer(block,
                                       layer_nums[2],
                                       in_channel=in_channels[2],
                                       out_channel=out_channels[2],
                                       stride=strides[2],
                                       use_se=self.use_se,
                                       se_block=self.se_block)
        self.layer4 = self._make_layer(block,
                                       layer_nums[3],
                                       in_channel=in_channels[3],
                                       out_channel=out_channels[3],
                                       stride=strides[3],
                                       use_se=self.use_se,
                                       se_block=self.se_block)

        self.mean = ops.ReduceMean(keep_dims=True)
```

```
        self.flatten = nn.Flatten()
        self.end_point = _fc(out_channels[3], num_classes, use_se=self.use_se)

    def _make_layer(self, block, layer_num, in_channel, out_channel, stride,
use_se=False, se_block=False):

        layers = []

        resnet_block = block(in_channel, out_channel, stride=stride, use_se=use_se)
        layers.append(resnet_block)
        if se_block:
            for _ in range(1, layer_num - 1):
                resnet_block = block(out_channel, out_channel, stride=1, use_se=use_se)
                layers.append(resnet_block)
            resnet_block = block(out_channel, out_channel, stride=1, use_se=use_se,
se_block=se_block)
            layers.append(resnet_block)
        else:
            for _ in range(1, layer_num):
                resnet_block = block(out_channel, out_channel, stride=1, use_se=use_se)
                layers.append(resnet_block)
        return nn.SequentialCell(layers)

    def construct(self, x):
        if self.use_se:
            x = self.conv1_0(x)
            x = self.bn1_0(x)
            x = self.relu(x)
            x = self.conv1_1(x)
            x = self.bn1_1(x)
            x = self.relu(x)
            x = self.conv1_2(x)
        else:
            x = self.conv1(x)
        x = self.bn1(x)
        x = self.relu(x)
        if self.res_base:
            x = self.pad(x)
        c1 = self.maxpool(x)

        c2 = self.layer1(c1)
        c3 = self.layer2(c2)
        c4 = self.layer3(c3)
        c5 = self.layer4(c4)

        out = self.mean(c5, (2, 3))
        out = self.flatten(out)
        out = self.end_point(out)
```

```
        return out


def se_resnet50(class_num=1001,phase="train"):

    return ResNet(ResidualBlock,
                  [3, 4, 6, 3],
                  [64, 256, 512, 1024],
                  [256, 512, 1024, 2048],
                  [1, 2, 2, 2],
                  class_num,
                  use_se=True)
```

- 模型训练

```
# 设置优化器
if config.lr_scheduler == 'step':
    lr = lr_steps(0, lr_init=config.lr_init, lr_max=config.lr_max,
warmup_epochs=config.warmup_epochs,
                  total_epochs=config.max_epoch, steps_per_epoch=batch_num)
# optimizer
opt = Momentum(params=get_param_groups(net),
               learning_rate=Tensor(lr),
               momentum=config.momentum,
               weight_decay=config.weight_decay,
               loss_scale=config.loss_scale)




# 开始训练
loss = nn.SoftmaxCrossEntropyWithLogits(sparse=True, reduction='mean')
model = Model(net, loss_fn=loss, optimizer=opt, metrics={"Accuracy": Accuracy()},
                  amp_level="O2", keep_batchnorm_fp32=False,loss_scale_manager=None)


# 定义回调函数
time_cb = TimeMonitor(data_size=batch_num)
loss_cb = LossMonitor(batch_num)
epoch_per_eval = {"epoch": [], "acc": []}
eval_cb = EvalCallBack(model, val_dataset, 2, epoch_per_eval)  #每个epoch都评估一下
callbacks = [time_cb, loss_cb,eval_cb]
if config.rank_save_ckpt_flag:
    ckpt_config = CheckpointConfig(save_checkpoint_steps=config.ckpt_interval *
config.steps_per_epoch,
                                   keep_checkpoint_max=config.keep_checkpoint_max)
    save_ckpt_path = os.path.join(config.outputs_dir, 'ckpt_' + str(config.rank) + '/')
```

```
    print(save_ckpt_path)
    ckpt_cb = ModelCheckpoint(config=ckpt_config,
                              directory=save_ckpt_path,
                              prefix='{}'.format(config.rank))
    callbacks.append(ckpt_cb)

#进行模型训练
model.train(config.max_epoch, dataset, callbacks=callbacks)
```

使用命令行运行，指令如下：

```
python train.py
```

8.5.3 模型验证

模型验证是测试模型鲁棒性关键的一步。在训练过程中，模型会不断拟合训练数据，同时在验证数据集上不断验证，并保留在验证集上最佳准确率的模型权重，为了测试模型的鲁棒性，需要一批新的数据对模型进行测试，测试代码如下所示：

```
# Copyright 2020 Huawei Technologies Co., Ltd
#
# Licensed under the Apache License, Version 2.0 (the "License");
# you may not use this file except in compliance with the License.
# You may obtain a copy of the License at
#
# http://www.apache.org/licenses/LICENSE-2.0
#
# Unless required by applicable law or agreed to in writing, software
# distributed under the License is distributed on an "AS IS" BASIS,
# WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
# See the License for the specific language governing permissions and
# limitations under the License.
# ============================================================================
"""Eval"""
import os
import time
import datetime
import random
import mindspore
import glob
import numpy as np
import mindspore.nn as nn
```

```python
from mindspore import Tensor, context
from mindspore.communication.management import init, get_rank, get_group_size
from mindspore.train.model import Model
from mindspore.train.serialization import load_checkpoint, load_param_into_net
from mindspore.ops import operations as P
from mindspore.ops import functional as F
from mindspore.common import dtype as mstype
from mindspore.train.callback import ModelCheckpoint, CheckpointConfig, LossMonitor,
TimeMonitor,Callback

from src.utils.logging import get_logger
from src.seresnet import se_resnet50
from src.dataset import vgg_create_dataset
from src.dataset import classification_dataset

from model_utils.moxing_adapter import config
from model_utils.moxing_adapter import moxing_wrapper
from model_utils.device_adapter import get_device_id, get_rank_id, get_device_num


class ParameterReduce(nn.Cell):
    """ParameterReduce"""
    def __init__(self):
        super(ParameterReduce, self).__init__()
        self.cast = P.Cast()
        self.reduce = P.AllReduce()

    def construct(self, x):
        one = self.cast(F.scalar_to_array(1.0), mstype.float32)
        out = x * one
        ret = self.reduce(out)
        return ret


def get_top5_acc(top5_arg, gt_class):
    sub_count = 0
    for top5, gt in zip(top5_arg, gt_class):
        if gt in top5:
            sub_count += 1
    return sub_count


def modelarts_pre_process():
    '''modelarts pre process function.'''
    def unzip(zip_file, save_dir):
        import zipfile
        s_time = time.time()
```

```
        if not os.path.exists(os.path.join(save_dir,
config.modelarts_dataset_unzip_name)):
            zip_isexist = zipfile.is_zipfile(zip_file)
            if zip_isexist:
                fz = zipfile.ZipFile(zip_file, 'r')
                data_num = len(fz.namelist())
                print("Extract Start...")
                print("unzip file num: {}".format(data_num))
                data_print = int(data_num / 100) if data_num > 100 else 1
                i = 0
                for file in fz.namelist():
                    if i % data_print == 0:
                        print("unzip percent: {}%".format(int(i * 100 / data_num)),
flush=True)
                    i += 1
                    fz.extract(file, save_dir)
                print("cost time: {}min:{}s.".format(int((time.time() - s_time) / 60),
                                                     int(int(time.time() - s_time) %
60)))
                print("Extract Done.")
            else:
                print("This is not zip.")
        else:
            print("Zip has been extracted.")

    if config.need_modelarts_dataset_unzip:
        zip_file_1 = os.path.join(config.data_path, config.modelarts_dataset_unzip_name
+ ".zip")
        save_dir_1 = os.path.join(config.data_path)

        sync_lock = "/tmp/unzip_sync.lock"

        # Each server contains 8 devices as most.
        if config.device_target == "GPU":
            init()
            device_id = get_rank()
            device_num = get_group_size()
        elif config.device_target == "Ascend":
            device_id = get_device_id()
            device_num = get_device_num()
        else:
            raise ValueError("Not support device_target.")

        # Each server contains 8 devices as most.
        if device_id % min(device_num, 8) == 0 and not os.path.exists(sync_lock):
            print("Zip file path: ", zip_file_1)
            print("Unzip file save dir: ", save_dir_1)
            unzip(zip_file_1, save_dir_1)
            print("===Finish extract data synchronization===")
```

```
            try:
                os.mknod(sync_lock)
            except IOError:
                pass

        while True:
            if os.path.exists(sync_lock):
                break
            time.sleep(1)

        print("Device: {}, Finish sync unzip data from {} to {}.".format(device_id,
zip_file_1, save_dir_1))

    config.log_path = os.path.join(config.output_path, config.log_path)


@moxing_wrapper(pre_process=modelarts_pre_process)
def run_eval():
    """run eval"""
    config.per_batch_size = config.batch_size
    config.image_size = list(map(int, config.image_size.split(',')))
    config.rank = get_rank_id()
    config.group_size = get_device_num()
    config.device_id = get_device_id()

    _enable_graph_kernel = config.device_target == "GPU"
    context.set_context(mode=context.PYNATIVE_MODE, enable_graph_kernel=False,
                        device_target=config.device_target, save_graphs=False)
    if os.getenv('DEVICE_ID') == str(config.device_id):
        pass
    else:
        if os.getenv('DEVICE_ID', "not_set").isdigit() and config.device_target ==
"Ascend":
            context.set_context(device_id=int(os.getenv('DEVICE_ID')))

    config.outputs_dir = os.path.join(config.log_path,
                                      datetime.datetime.now().strftime('%Y-%m-
%d_time_%H_%M_%S'))

    config.logger = get_logger(config.outputs_dir, config.rank)
    config.logger.save_args(config)

    if config.dataset == "garbage_photos":
        net = se_resnet50(class_num=config.num_classes,phase="test")
        loss = nn.SoftmaxCrossEntropyWithLogits(sparse=True, reduction='mean')
        model = Model(net, loss_fn=loss, metrics={'acc'})
        param_dict = load_checkpoint(config.pre_trained)
        load_param_into_net(net, param_dict)
        net.set_train(False)
```

```
        dataset = vgg_create_dataset(config.data_dir, config.image_size,
config.per_batch_size, training=False,valid=False)
        res = model.eval(dataset)
        print("result: ", res)
```

使用命令行运行，指令如下：

```
python eval.py
```

验证acc为：

```
res = model.eval(dataset)
print("result: ", res)
```

```
result:  {'acc': 0.9206349206349206}
```

8.5.4 模型导出

首先将MindSpore的ckpt模型转为AIR模型

```
import os
import numpy as np
from mindspore import Tensor, context, export
import mindspore as ms
from mindspore import load_checkpoint, load_param_into_net
from src.seresnet import se_resnet50
context.set_context(mode=context.GRAPH_MODE, device_target=device_target)

def run_export(ckpt_file, save_filename, num_classes, image_size, device_target,
device_id):
    # 初始化模型
    net = se_resnet50(class_num=num_classes)
    # 加载权重参数
    load_param_into_net(net, load_checkpoint(ckpt_file), strict_load=True)
    # 设置为eval模式
    net.set_train(False)
    # 构建伪输入
    input_data = Tensor(np.zeros([1, 3, image_size[0], image_size[1]]), ms.float32)
    # 导出模型
    export(net, input_data, file_name=save_filename, file_format="AIR")
# 执行导出函数
run_export(ckpt_file="./pretrained/senet50_acc70.ckpt",
           save_filename="garbage_deploy",
           num_classes=40,
           image_size=(224,224),
```

```
            device_target="Ascend",
            device_id=0)
```

将AIR模型转为om模型，彻底摆脱MindSpore计算框架的同时，充分发挥Ascend硬件的计算能力。

```
export install_path=/usr/local/Ascend/ascend-toolkit/latest
export
PATH=/usr/local/python3.7.5/bin:${install_path}/atc/ccec_compiler/bin:${install_path}/a
tc/bin:$PATH
export PYTHONPATH=${install_path}/atc/python/site-packages:$PYTHONPATH
export
LD_LIBRARY_PATH=${install_path}/atc/lib64:${install_path}/acllib/lib64:$LD_LIBRARY_PATH
export ASCEND_OPP_PATH=${install_path}/opp
export ASCEND_AICPU_PATH=/usr/local/Ascend/ascend-toolkit/latest

/usr/local/Ascend/ascend-toolkit/latest/atc/bin/atc --framework=1 \
      --model="garbage_deploy.mindir" \
            --output="garbage_deploy" \
            --input_format=NCHW \
            --input_shape="image:1,3:224,224" \
            --log=error \
            --soc_version=Ascend310
```

8.5.5 模型部署

对于导出模型，需要打包成推理应用服务，构建从输入图片到输出类别概率端到端的服务。

首先构建推理引擎类，代码如下所示：

```
import numpy as np
import struct
import time
import acl

ACL_MEM_MALLOC_HUGE_FIRST = 0
ACL_MEMCPY_HOST_TO_DEVICE = 1
ACL_MEMCPY_DEVICE_TO_HOST = 2
NPY_BYTE = 1


class ACLEngine(object):
    def __init__(self, model_path, class_map, device_id=0):
        self.device_id = device_id
        self.context = None
        self.model_id = None
        self.model_desc = None
        self.load_input_dataset, self.load_output_dataset = None, None
```

```
        self.input_data, self.output_data = None, None
        self.class_map = class_map
        self.num_classes = len(class_map)

        # --------------- pyACL初始化 ---------------
        print('Start init resource')
        self._init_resource()
        self._load_model(model_path)
        self._prepare_inputs()

    def inference(self, img_path):
        """ 模型推理及后处理模块 """
        # 1.读取并预处理图片
        img = process_image(img_path)
        # 2.准备模型推理的输入数据，运行模式默认为运行模式为ACL_HOST，当前实例代码中模型只有一个输
入。

        bytes_data = img.tobytes()
        np_ptr = acl.util.bytes_to_ptr(bytes_data)

        start_time = time.time()
        # 将图片数据从Host传输到Device。
        ret = acl.rt.memcpy(self.input_data[0]["buffer"], self.input_data[0]["size"],
np_ptr,
                            self.input_data[0]["size"], ACL_MEMCPY_HOST_TO_DEVICE)

        # 3.执行模型推理。
        # self.model_id表示模型ID，在模型加载成功后，会返回标识模型的ID。
        ret = acl.mdl.execute(self.model_id, self.load_input_dataset,
self.load_output_dataset)

        # 4.处理模型推理的输出数据，输出置信度的类别编号。
        inference_result = []
        for i, item in enumerate(self.output_data):
            buffer_host, ret = acl.rt.malloc_host(self.output_data[i]["size"])
            # 将推理输出数据从Device传输到Host。
            ret = acl.rt.memcpy(buffer_host, self.output_data[i]["size"],
self.output_data[i]["buffer"],
                                self.output_data[i]["size"], ACL_MEMCPY_DEVICE_TO_HOST)

            bytes_out = acl.util.ptr_to_bytes(buffer_host, self.output_data[i]["size"])
            data = np.frombuffer(bytes_out, dtype=np.byte)
            inference_result.append(data)

        tuple_st = struct.unpack(f"{self.num_classes}f",
bytearray(inference_result[0]))
        vals = np.array(tuple_st).flatten()
        top_k = vals.argsort()[-1:-6:-1]
        print("\n======== inference results: =============")
```

```
        for i, j in enumerate(top_k):
            print("top %d: class:[%s]: probability:[%f]" % (i, self.class_map[str(j)],
vals[j]))

        end_time = time.time()
        print('inference cost time: {:.1f}ms\n'.format((end_time-start_time)*1000))

    def release_resource(self):
        """ 资源释放模块 """
        self._unload_model()
        self._unload_picture()
        self._destroy_resource()
        print('Resource destroyed successfully')

    def _init_resource(self):
        # pyACL初始化
        ret = acl.init()

        # 运行管理资源申请
        # 指定运算的Device。
        self.device_id = 0
        ret = acl.rt.set_device(self.device_id)
        # 显式创建一个Context，用于管理Stream对象。
        self.context, ret = acl.rt.create_context(self.device_id)

    def _load_model(self, model_path):
        # 加载离线模型文件，返回标识模型的ID。
        self.model_id, ret = acl.mdl.load_from_file(model_path)

        # 根据加载成功的模型的ID，获取该模型的描述信息。
        self.model_desc = acl.mdl.create_desc()
        ret = acl.mdl.get_desc(self.model_desc, self.model_id)

    def _prepare_inputs(self):
        # 1.准备模型推理的输入数据集。
        # 创建aclmdlDataset类型的数据，描述模型推理的输入。
        self.load_input_dataset = acl.mdl.create_dataset()
        # 获取模型输入的数量。
        input_size = acl.mdl.get_num_inputs(self.model_desc)
        self.input_data = []
        # 循环为每个输入申请内存，并将每个输入添加到aclmdlDataset类型的数据中。
        for i in range(input_size):
            buffer_size = acl.mdl.get_input_size_by_index(self.model_desc, i)
            # 申请输入内存。
            buffer, ret = acl.rt.malloc(buffer_size, ACL_MEM_MALLOC_HUGE_FIRST)
            data = acl.create_data_buffer(buffer, buffer_size)
            _, ret = acl.mdl.add_dataset_buffer(self.load_input_dataset, data)
            self.input_data.append({"buffer": buffer, "size": buffer_size})
```

```
        # 2.准备模型推理的输出数据集。
        # 创建aclmdlDataset类型的数据，描述模型推理的输出。
        self.load_output_dataset = acl.mdl.create_dataset()
        # 获取模型输出的数量。
        output_size = acl.mdl.get_num_outputs(self.model_desc)
        self.output_data = []
        # 循环为每个输出申请内存，并将每个输出添加到aclmdlDataset类型的数据中。
        for i in range(output_size):
            buffer_size = acl.mdl.get_output_size_by_index(self.model_desc, i)
            # 申请输出内存。
            buffer, ret = acl.rt.malloc(buffer_size, ACL_MEM_MALLOC_HUGE_FIRST)
            data = acl.create_data_buffer(buffer, buffer_size)
            _, ret = acl.mdl.add_dataset_buffer(self.load_output_dataset, data)
            self.output_data.append({"buffer": buffer, "size": buffer_size})

    def _unload_model(self):
        # 卸载模型。
        ret = acl.mdl.unload(self.model_id)
        # 释放模型描述信息。
        if self.model_desc:
            ret = acl.mdl.destroy_desc(self.model_desc)
            self.model_desc = None
        # 释放Context。
        if self.context:
            ret = acl.rt.destroy_context(self.context)
            self.context = None

    def _unload_picture(self):
        # 释放输出资源，包括数据结构和内存。
        while self.output_data:
            item = self.output_data.pop()
            ret = acl.rt.free(item["buffer"])
        output_number = acl.mdl.get_dataset_num_buffers(self.load_output_dataset)
        for i in range(output_number):
            data_buf = acl.mdl.get_dataset_buffer(self.load_output_dataset, i)
            if data_buf:
                ret = acl.destroy_data_buffer(data_buf)
        ret = acl.mdl.destroy_dataset(self.load_output_dataset)

    def _destroy_resource(self):
        # 释放Device。
        ret = acl.rt.reset_device(self.device_id)
        # pyACL去初始化。
        ret = acl.finalize()
```

随后进行初始化并进行推理

```python
import json
with open("../data/label_list.json", 'r', encoding='utf-8') as json_path:
    class_map = json.load(json_path)

# 初始化推理引擎
engine = ACLEngine('garbage_deploy.om', class_map, device_id=0)
# 推理单张图片
engine.inference('./data/test_demo.jpg')
# 释放资源
engine.release_resource()
```

8.5.6 实验运行

目录说明

```
|- code
| |- deploy_model
| | |- garbage_deploy.air   # AIR模型
| | |- garbage_deploy.om    # om模型
| |- convert_air_to_om.sh     # AIR转om模型脚本
| |- deploy_app.py        # 推理部署服务脚本
| |- export.py          # ckpt导出AIR模型脚本
|- data
```

8.6 评分指标

70：完成垃圾分类模型训练，且精度不低于80%

80：完成垃圾分类模型部署，精度不低于80%的同时且推理时间不大于30ms

90：完成垃圾分类模型训练和部署，精度不低于85%推理时间不大于20ms

100：完成垃圾分类模型训练和部署，精度不低于90%推理时间不大于15ms